

计算机系统基础

程序的机器级表示（5）

王晶

jwang@ruc.edu.cn, 信息楼124

2025年11月



过程机制

- 传递控制
 - 到过程代码的开头
 - 回到返回点
- 参数传递
 - 输入参数
 - 返回值
- 存储空间管理
 - 执行期间动态分配
 - 返回时释放空间
- 通过机器指令实现上述过程

```
P (...) {  
    .  
    .  
    y = Q(x);  
    print(y)  
    .  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    .  
    .  
    return v[t];  
}
```

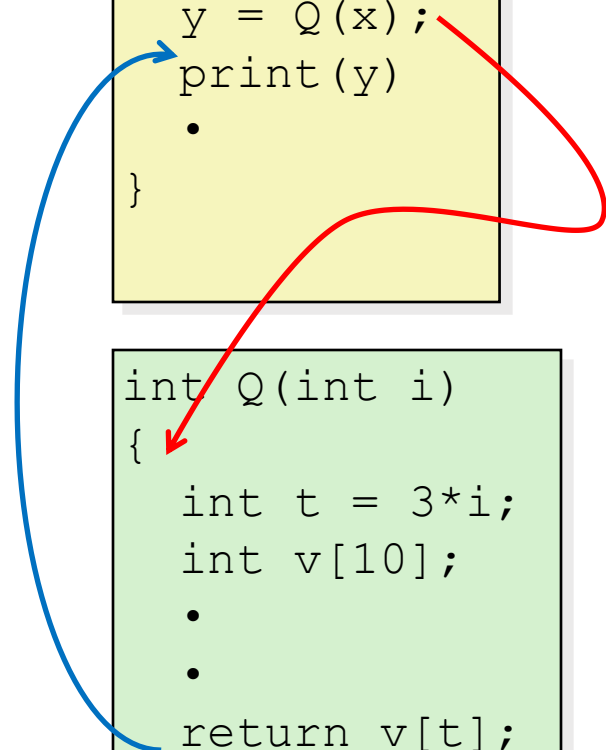


过程机制

- 传递控制
 - 到过程代码的开头
 - 回到返回点
- 参数传递
 - 输入参数
 - 返回值
- 存储空间管理
 - 执行期间动态分配
 - 返回时释放空间
- 通过机器指令实现上述过程

```
P (...) {  
    .  
    .  
    y = Q(x);  
    print(y)  
    .  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    .  
    .  
    return v[t];  
}
```





过程机制

- 传递控制
 - 到过程代码的开头
 - 回到返回点
- 参数传递
 - 输入参数
 - 返回值
- 存储空间管理
 - 执行期间动态分配
 - 返回时释放空间
- 通过机器指令实现上述过程

```
P (...) {  
    .  
    .  
    y = Q(x);  
    print(y)  
    .  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    .  
    .  
    return v[t];  
}
```



过程机制

- 传递控制
 - 到过程代码的开头
 - 回到返回点
- 参数传递
 - 输入参数
 - 返回值
- 存储空间管理
 - 执行期间动态分配
 - 返回时释放空间
- 通过机器指令实现上述过程

```
P (...) {  
    .  
    .  
    y = Q(x);  
    print(y)  
    .  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    .  
    .  
    return v[t];  
}
```



目录

- 过程
 - 执行原理
 - 栈结构
 - 调用约定
 - 传递控制
 - 传递数据
 - 管理局部数据
 - 递归



过程控制流

- 用栈完成过程的调用和返回
- 过程调用: **call label**
 - 返回地址压栈
 - 跳转到 $\left\{ \begin{array}{l} \text{label (direct)} \\ \text{*operand (indirect)} \end{array} \right.$
- 返回地址:
 - 调用指令的下一条指令
 - Example from disassembly
- 过程返回: **ret**
 - 从栈中弹出返回地址
 - 跳转到弹出地址处

```
push retaddr  
jmp callee
```

```
pop retaddr  
jmp retaddr
```

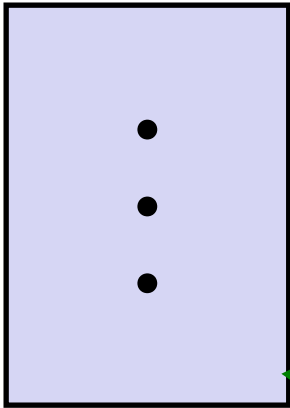


Control Flow Example #1

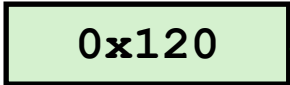
```
00000000000400540 <multstore>:
.
.
400544: callq 400550 <mult2>
400549: mov   %rax, (%rbx)
.
.
```

```
00000000000400550 <mult2>:
400550: mov   %rdi,%rax
.
.
400557: retq
```

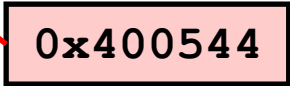
0x130
0x128
0x120



%rsp



%rip

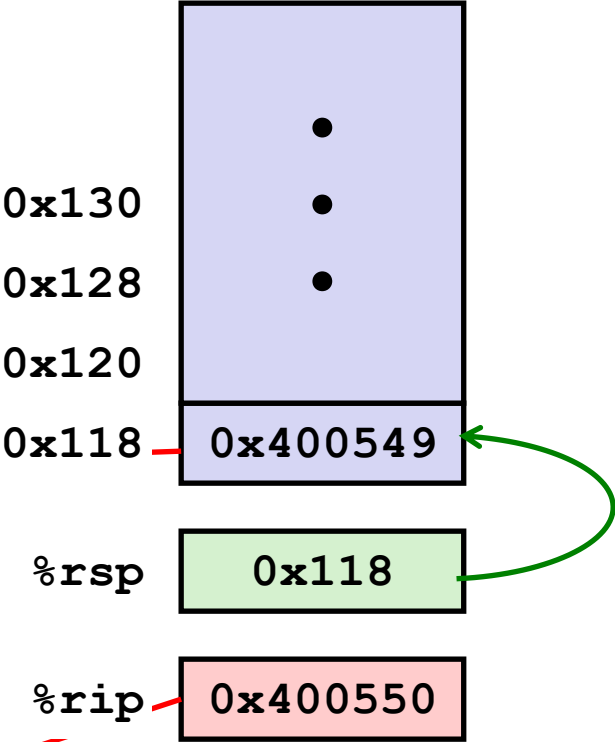




Control Flow Example #2

```
00000000000400540 <multstore>:  
.  
.  
400544: callq 400550 <mult2>  
400549: mov %rax, (%rbx) ←  
.  
.
```

```
00000000000400550 <mult2>:  
400550: mov %rdi, %rax ←  
.  
.  
400557: retq
```

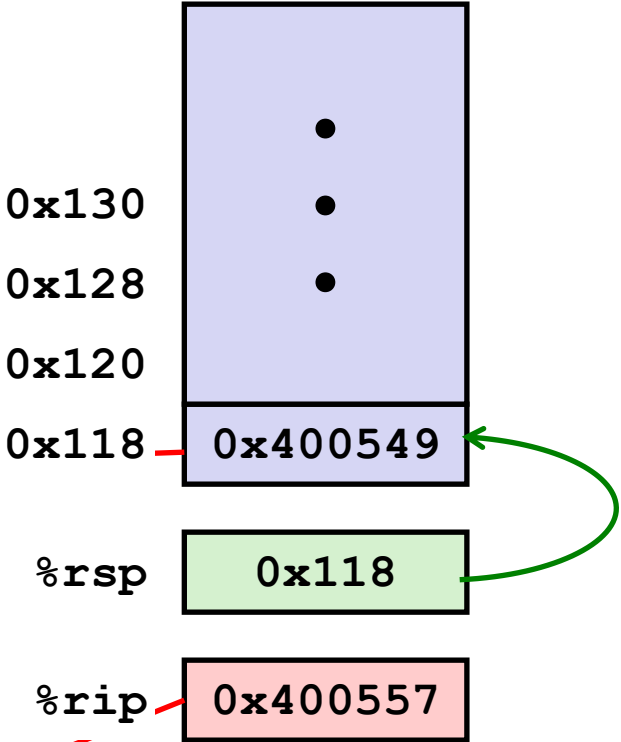




Control Flow Example #3

```
00000000000400540 <multstore>:
.
.
400544: callq 400550 <mult2>
400549: mov   %rax, (%rbx)
.
.
```

```
00000000000400550 <mult2>:
400550: mov   %rdi,%rax
.
.
400557: retq
```





Control Flow Example #4

```
00000000000400540 <multstore>:  
.  
.  
400544: callq 400550 <mult2>  
400549: mov    %rax, (%rbx)  
.  
.
```

```
00000000000400550 <mult2>:  
400550: mov    %rdi,%rax  
.  
.  
400557: retq
```

0x130

0x128

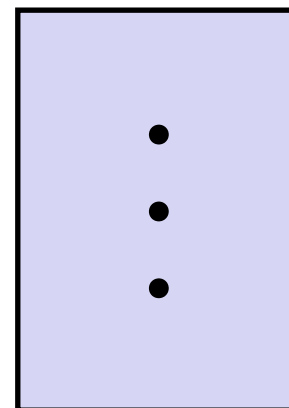
0x120

%rsp

0x120

%rip

0x400549





课堂练习

- 下面的代码片段经常出现在库函数的编译版本中：

```
call next  
next:  
popl %eax
```

- A) `eax`被设置成什么值？
- B) 解释为什么这个调用没有与之匹配的`ret`指令
- C) 这段代码完成了什么功能？



Today

- 过程
 - 执行原理
 - 栈结构
 - 调用约定
 - 传递控制
 - 传递数据
 - 管理局部数据
 - 递归



Call Chain Example

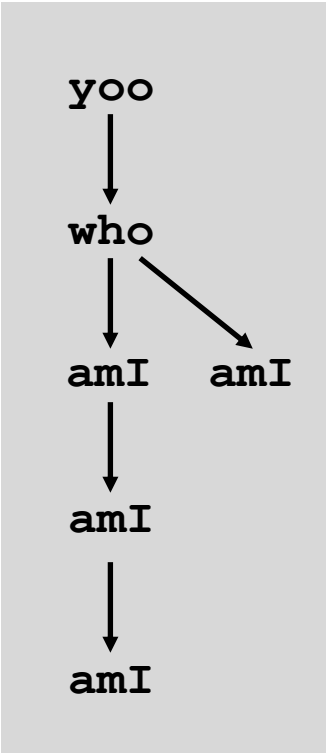
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```

```
who (...)  
{  
  . . .  
  amI ();  
  . . .  
  amI ();  
  . . .  
}
```

```
amI (...)  
{  
  .  
  .  
  amI ();  
  .  
  .  
}
```

Procedure amI () is recursive

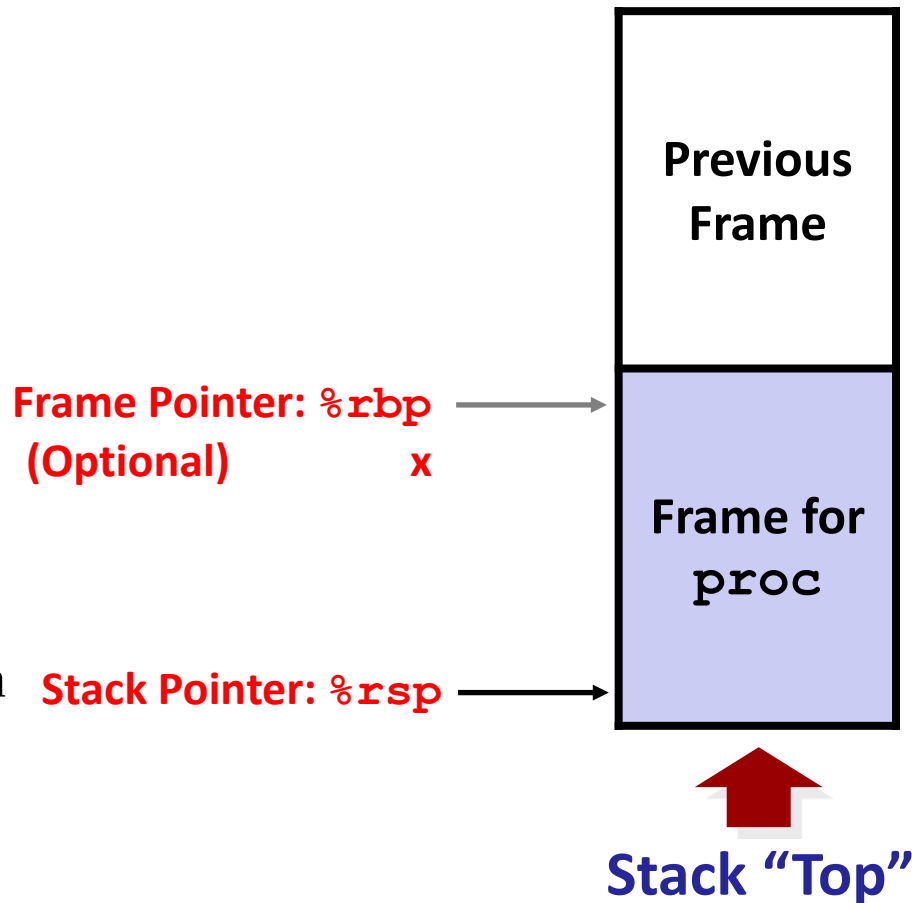
Example
Call Chain





栈帧

- 栈帧的内容
 - 返回信息
 - 本地存储
 - 临时变量
- 栈帧的管理
 - 进入调用程序时分配
 - “Set-up” code
 - Includes push by **call** instruction
 - 退出调用程序时释放
 - “Finish” code
 - Includes pop by **ret** instruction

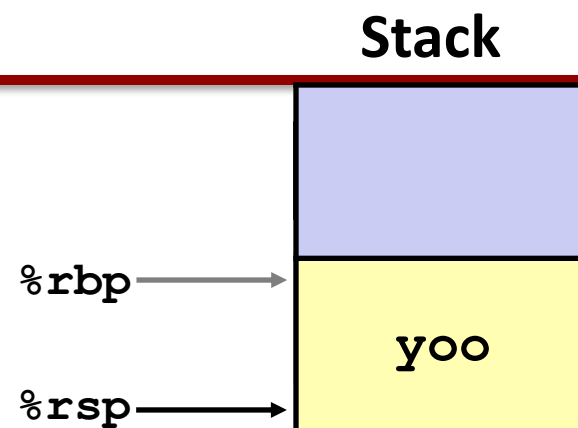
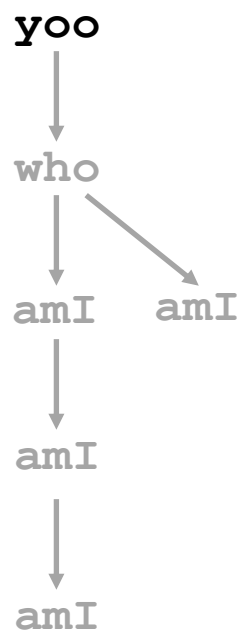




Example

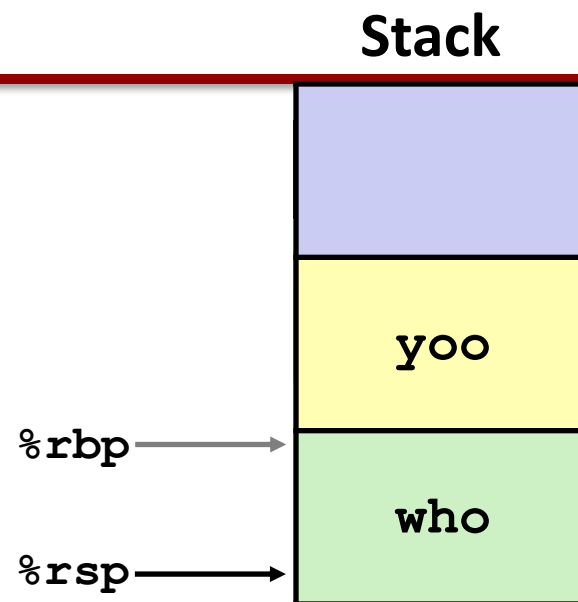
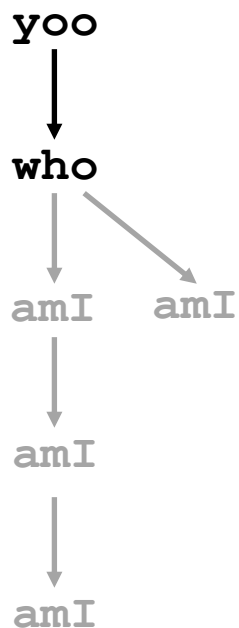
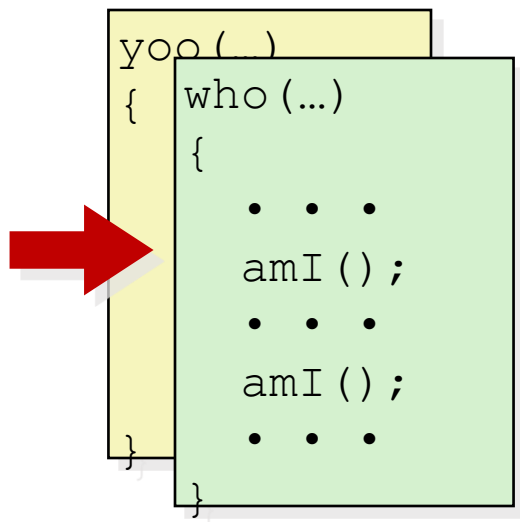


```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```



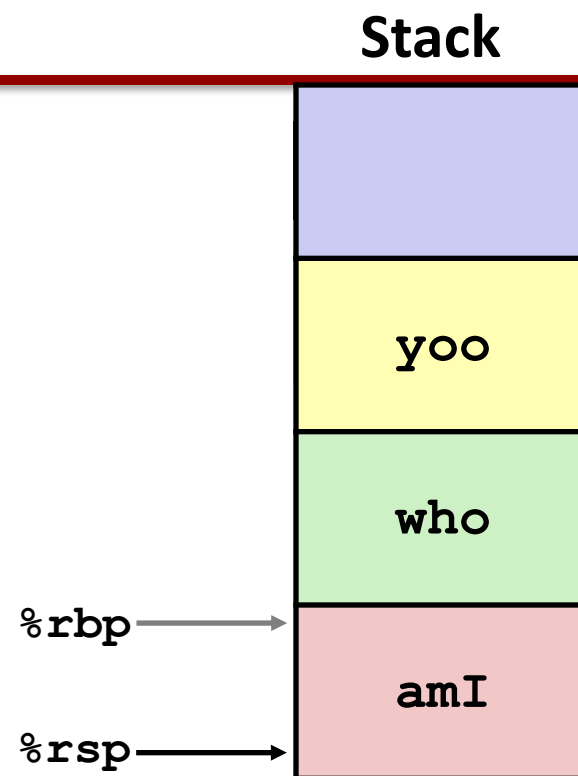
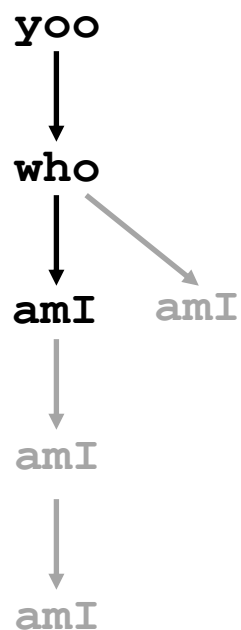
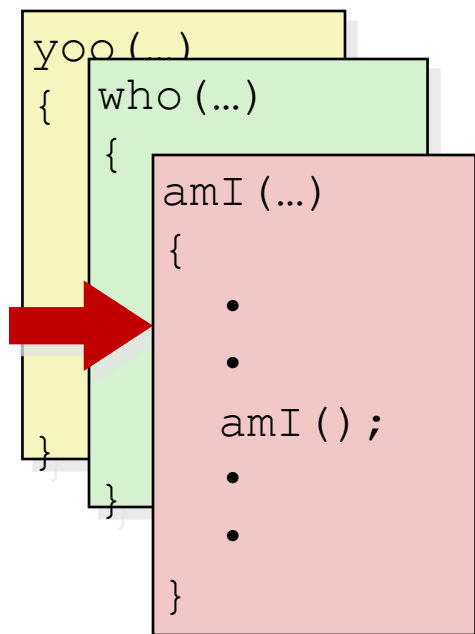


Example



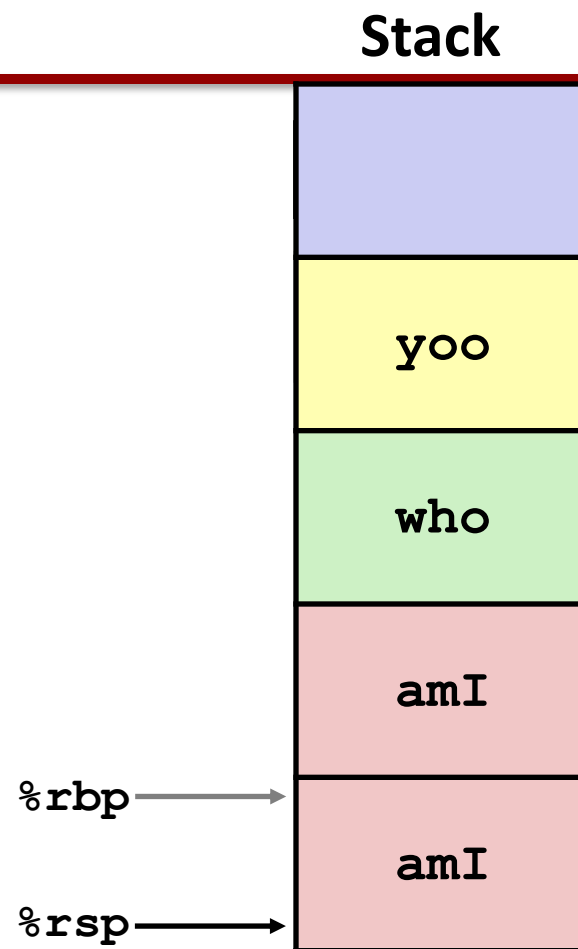
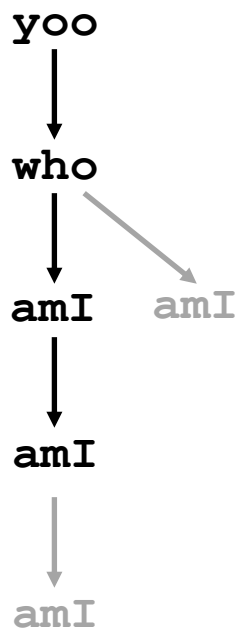
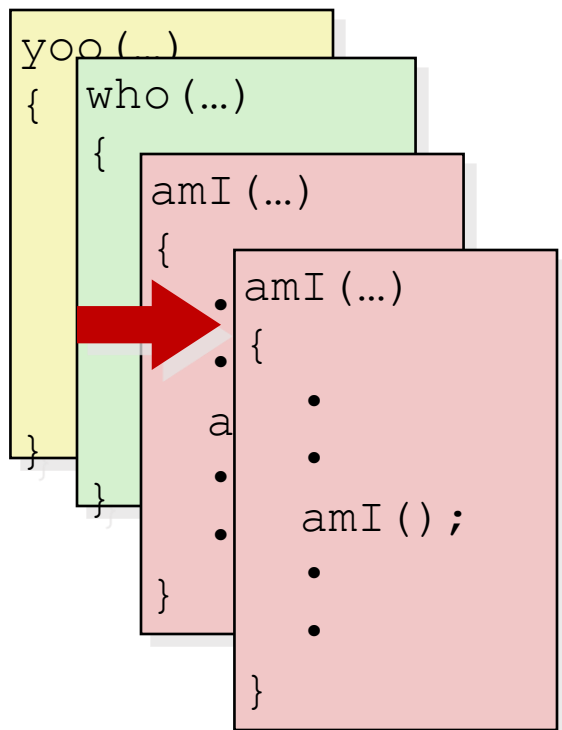


Example



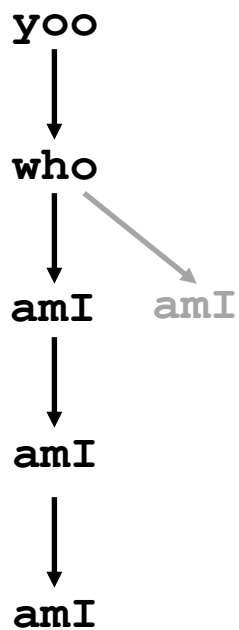
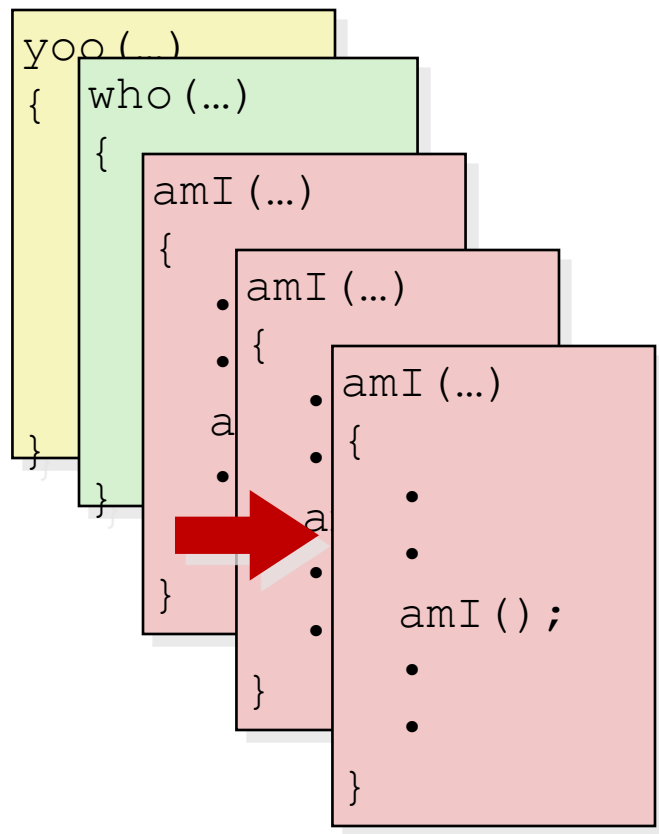


Example





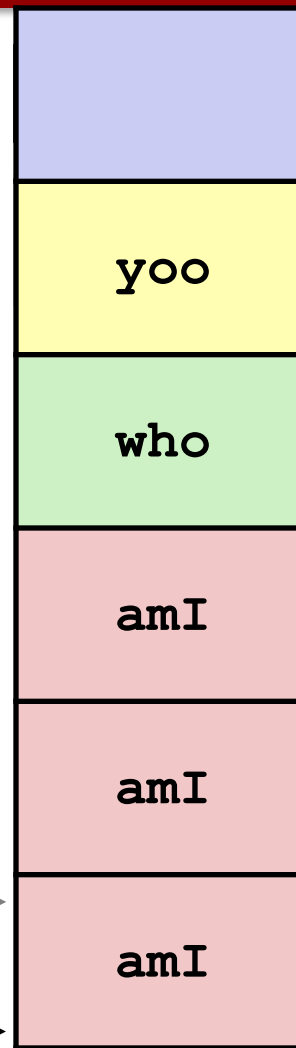
Example



Stack

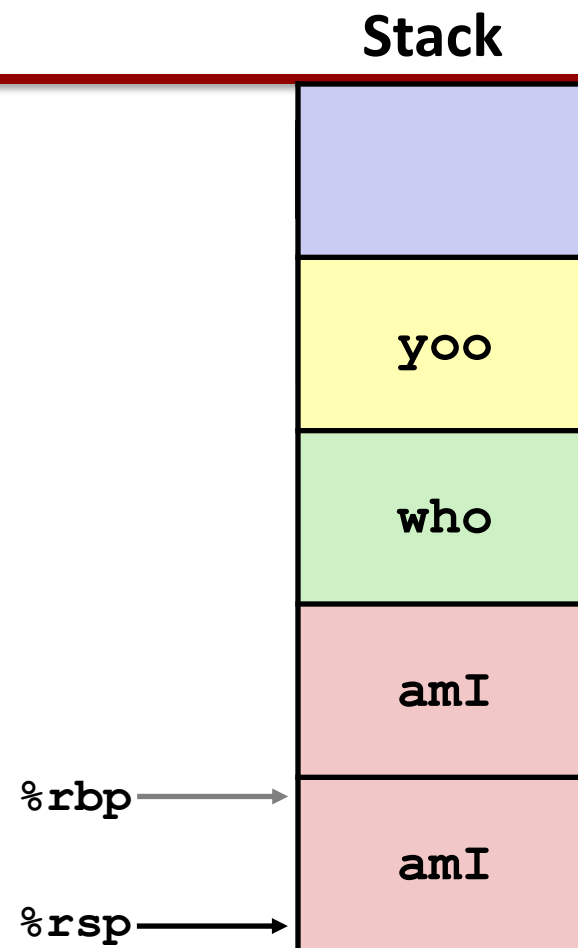
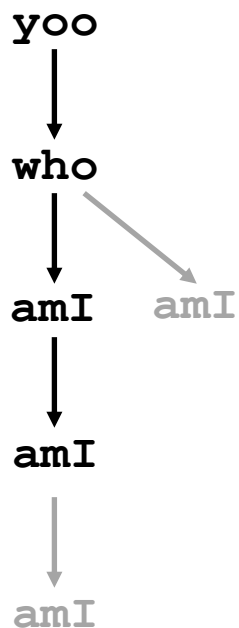
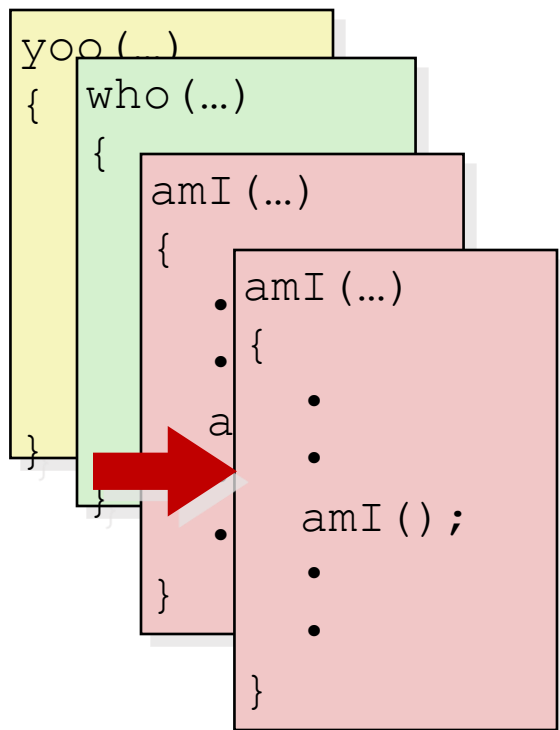
`%rbp` →

`%rsp` →



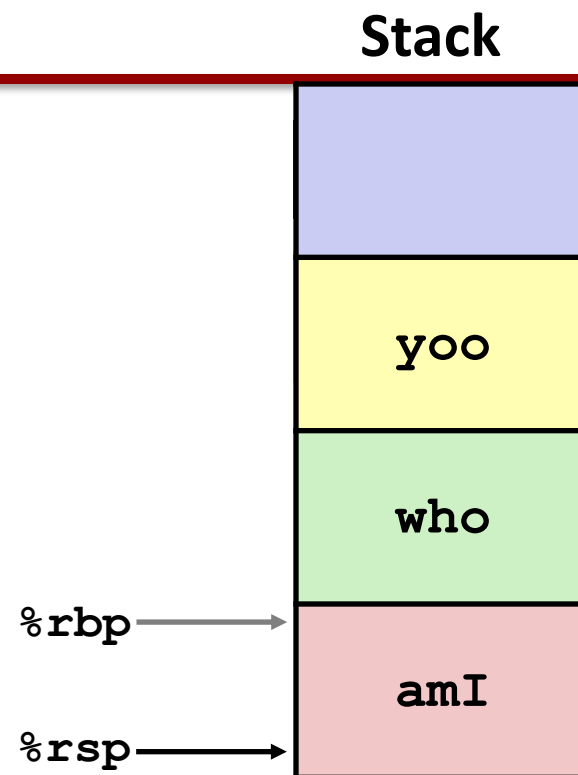
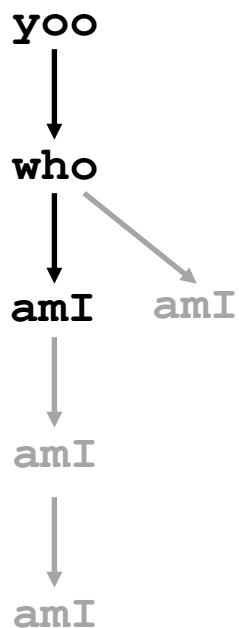
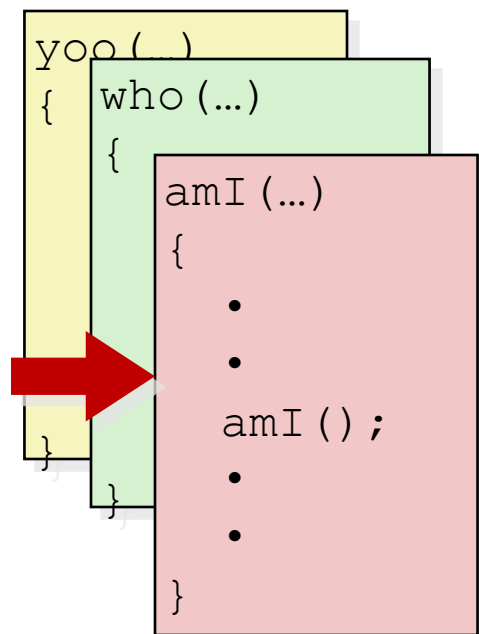


Example



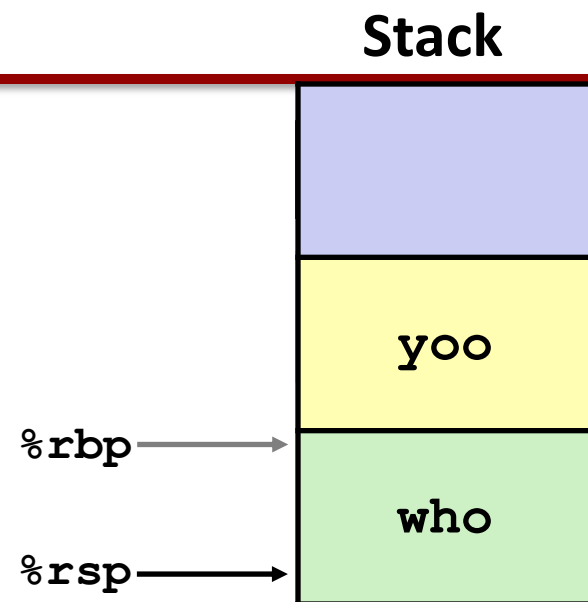
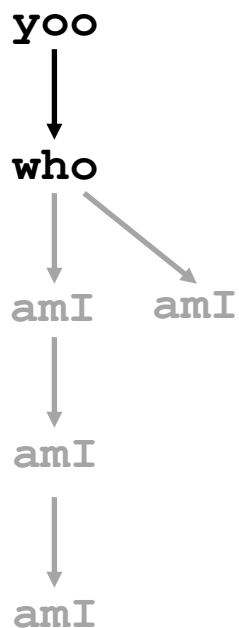
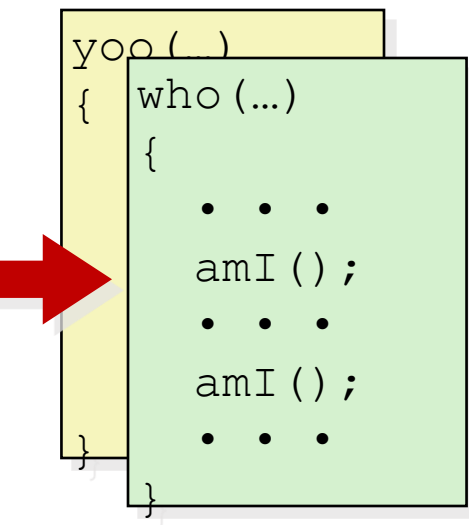


Example



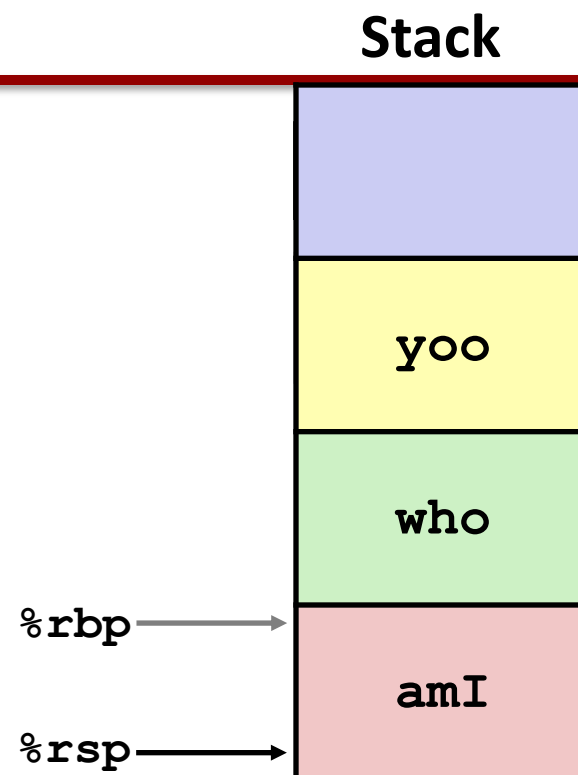
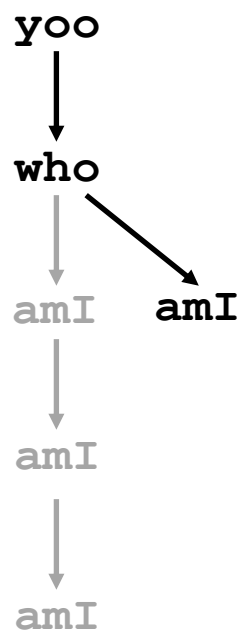
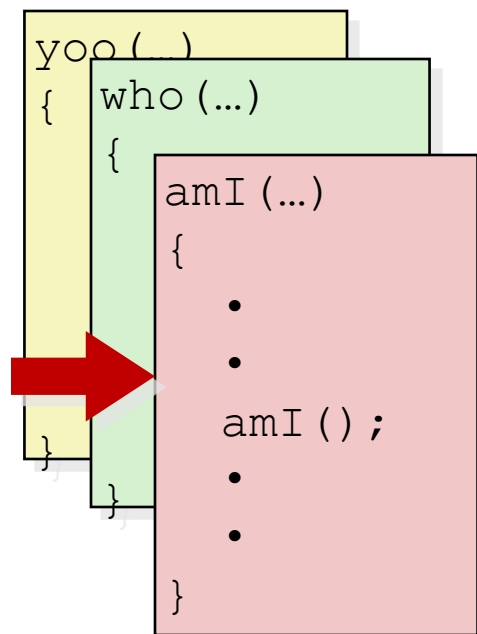


Example



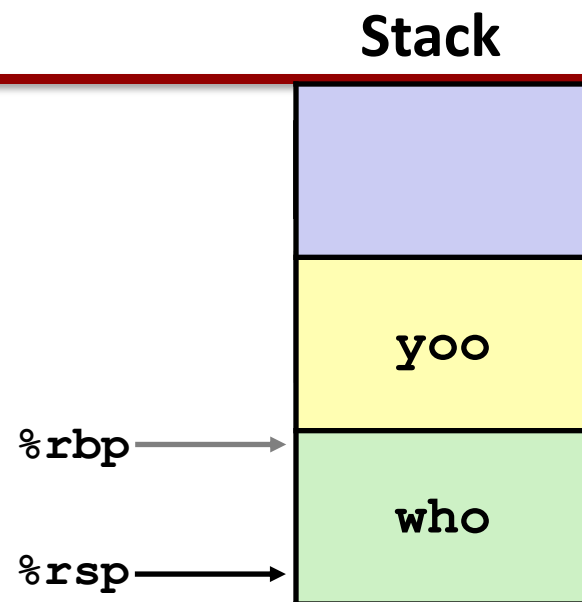
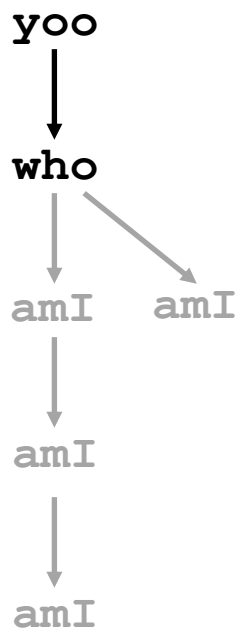
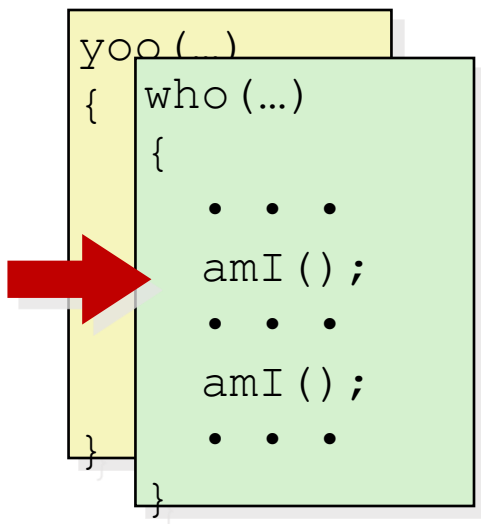


Example





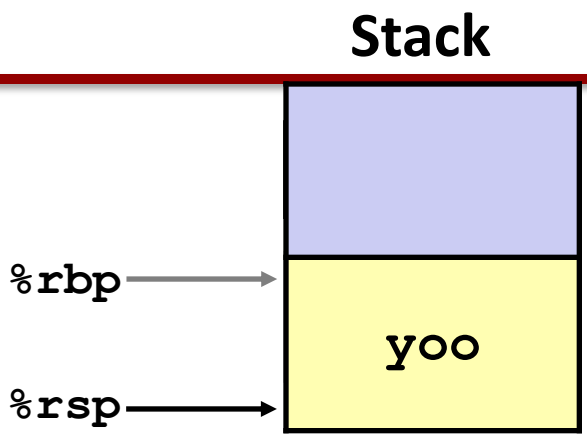
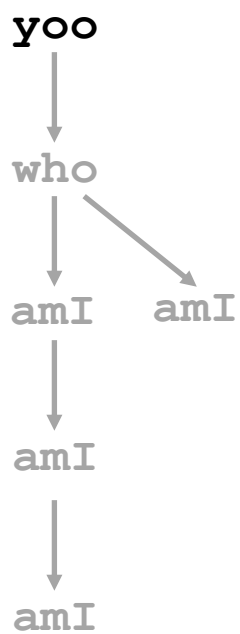
Example





Example

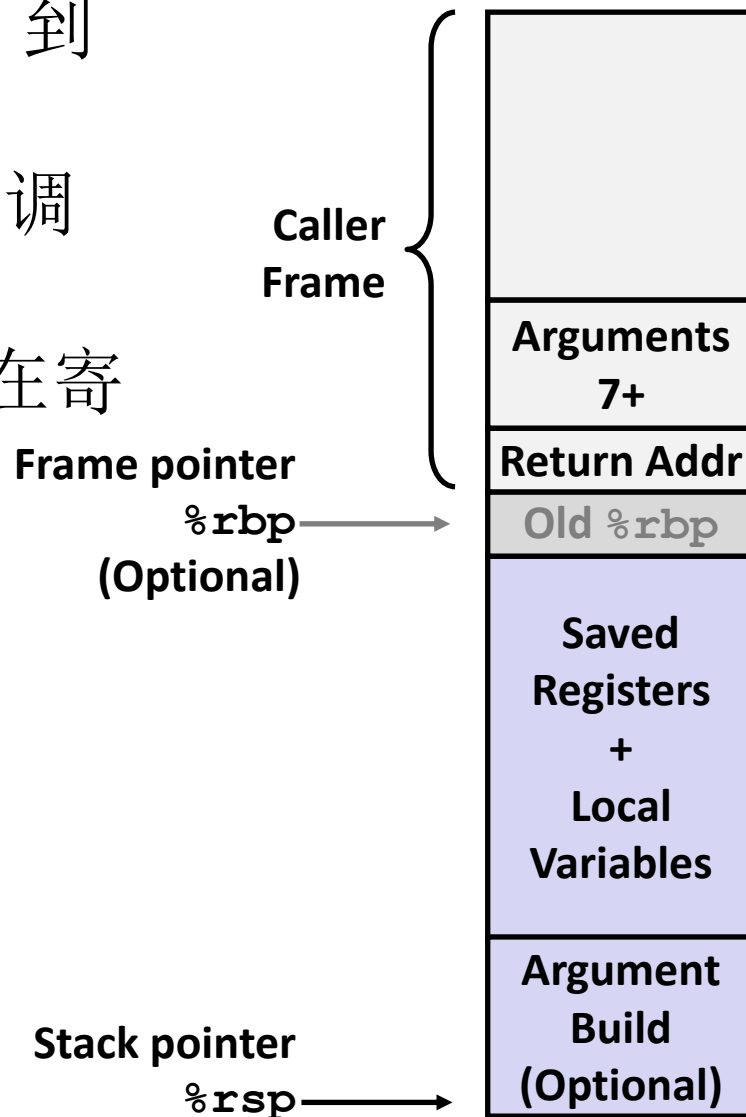
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```





x86-64/Linux 栈帧

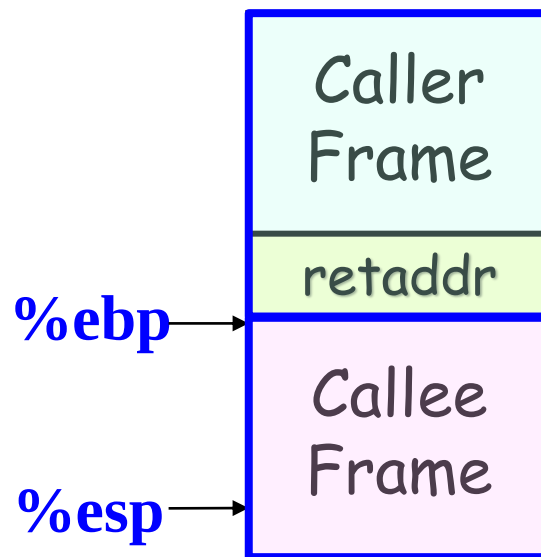
- 当期Callee栈帧（“顶部”到底部）
 - “Argument build:”即将调用的函数的参数
 - 局部变量，如果无法保留在寄存器中
 - 保存的寄存器上下文
 - 旧帧指针（可选）
- Caller栈帧
 - 返回地址
 - 由call指令压栈
 - 传递的参数





栈帧结构

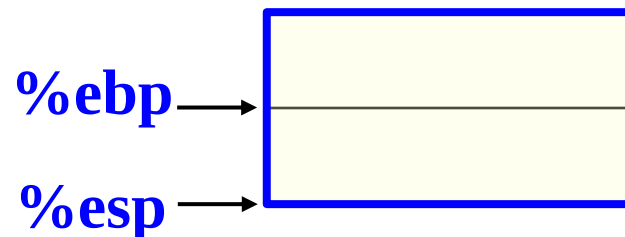
- 主函数、子函数共享同一个system stack
- call: save *return address* in the stack
- ret: pop *return address* from stack
 - The end of caller's stack frame





Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame

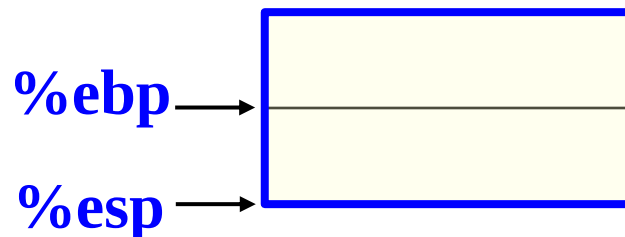




Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee





Frame Chain

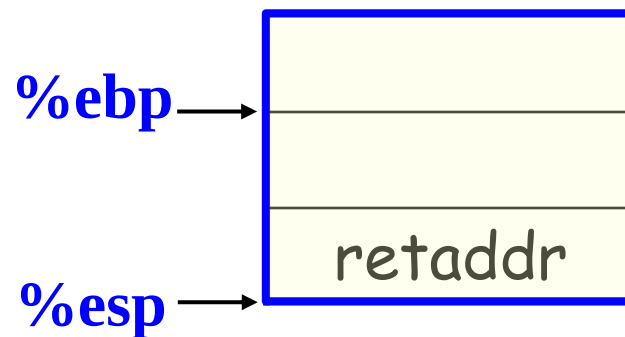
- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

3. mov %esp, %ebp





Frame Chain

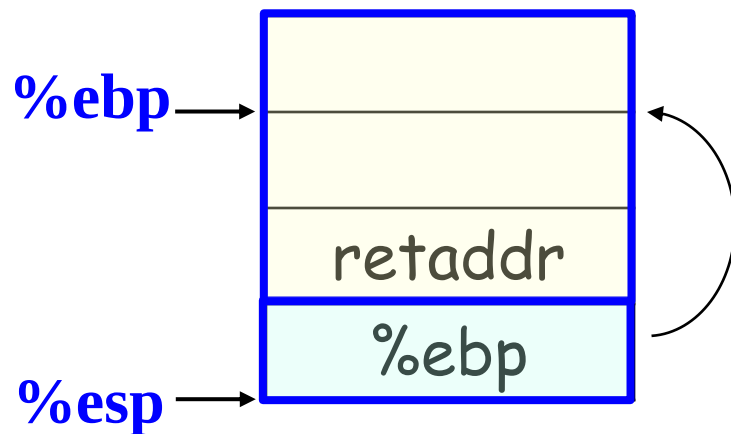
- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

3. mov %esp, %ebp





Frame Chain

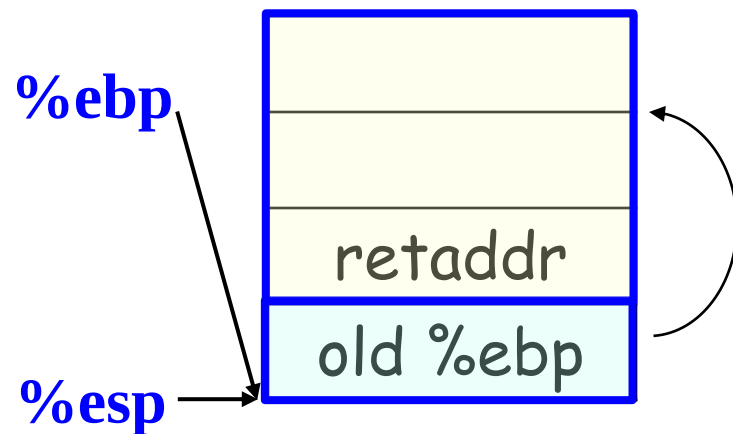
- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

3. mov %esp, %ebp





Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

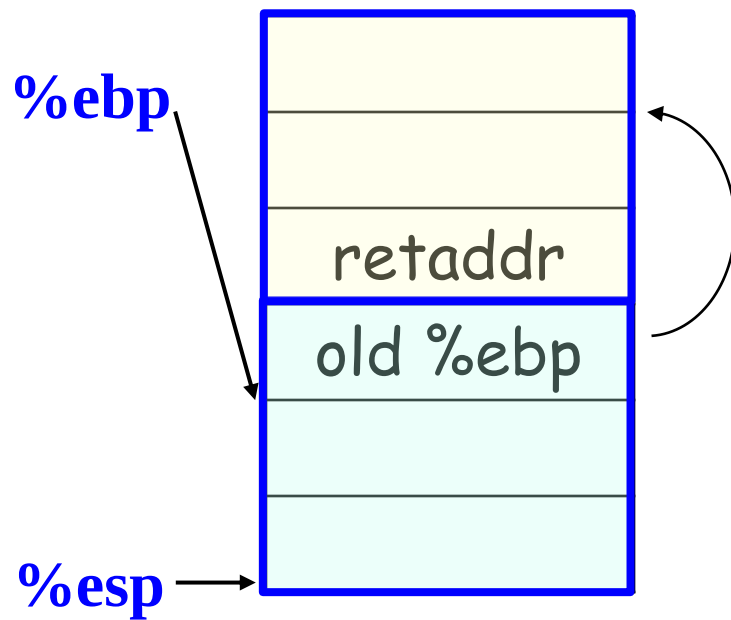
3. mov %esp, %ebp

...

n-2. mov %ebp, %esp

n-1. pop %ebp

n. ret





Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

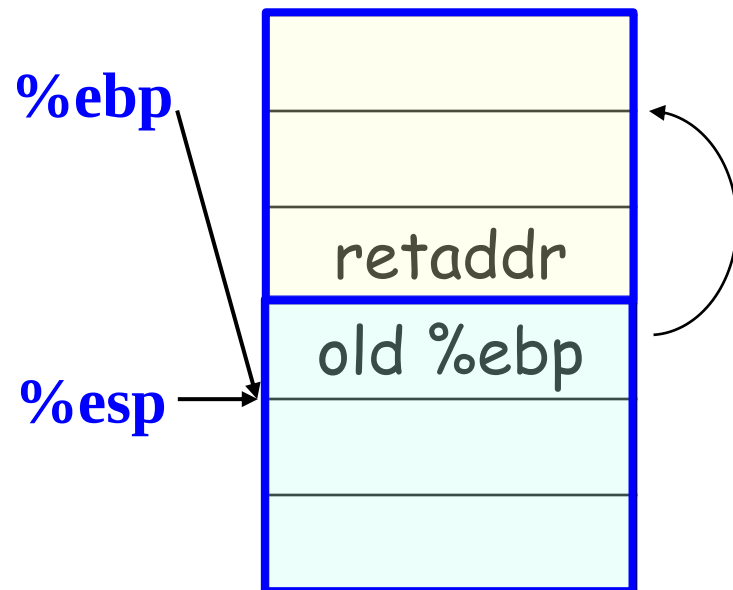
3. mov %esp, %ebp

...

n-2. mov %ebp, %esp

n-1. pop %ebp

n. ret





Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

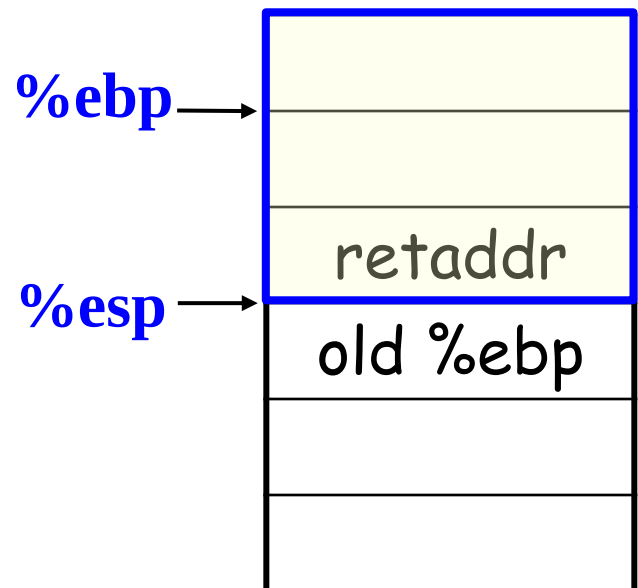
3. mov %esp, %ebp

...

n-2. mov %ebp, %esp

n-1. pop %ebp

n. ret





Frame Chain

- Pointers (%ebp/%esp) only delimit **topmost** frame
- Frames are chained

1. call callee

callee:

2. push %ebp

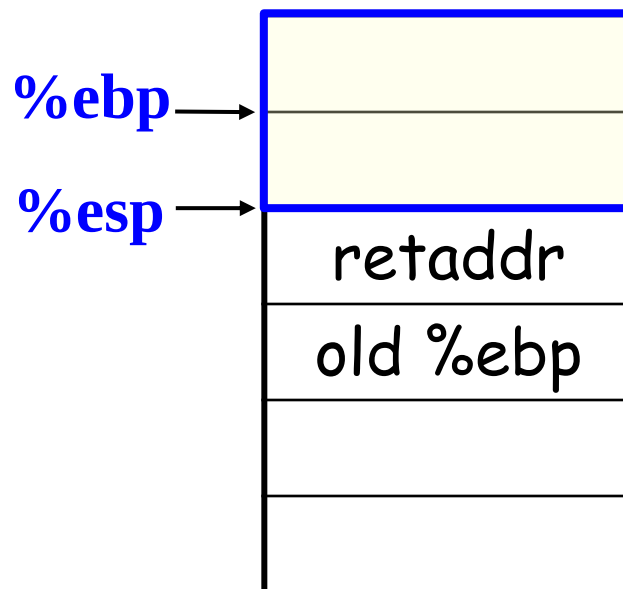
3. mov %esp, %ebp

...

n-2. mov %ebp, %esp

n-1. pop %ebp

n. ret





保存 Caller %ebp

- Instruction
 - **leave**
- Behavior description (by hardware)
 - Adjust %esp to callee %ebp
 - Pop *caller %ebp* from stack

leave = mov + pop

```
mov %ebp, %esp  
pop %ebp
```



call and ret的执行

//beginning of function sum

```
1. 08048394 <sum>:  
2. 8048394: 55                push  %ebp  
   . . .  
3. 80483a4:                ret  
   . . .
```

//call to sum from main

```
4. 80483dc: e8 b3 ff ff ff  call  8048394<sum>  
5. 80483e1: 83 c4 14        add   $0x14, %esp
```

Executing call		After call		After ret	
%eip	0x080483dc	%eip	0x08048394	%eip	0x080483e1
%esp	0xff9bc960	%esp	0xff9bc95c	%esp	0xff9bc960
ret	-	ret	0x080483e1	ret	-



Today

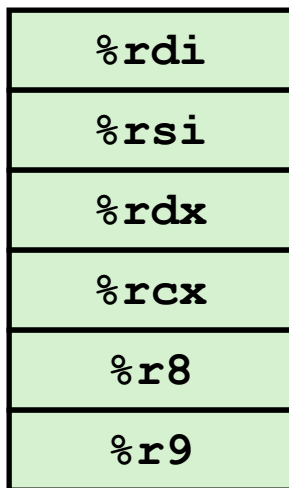
- 过程
 - 执行原理
 - 栈结构
 - 调用约定
 - 传递控制
 - 传递数据
 - 管理局部数据
- 递归



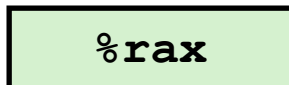
过程数据流

Registers

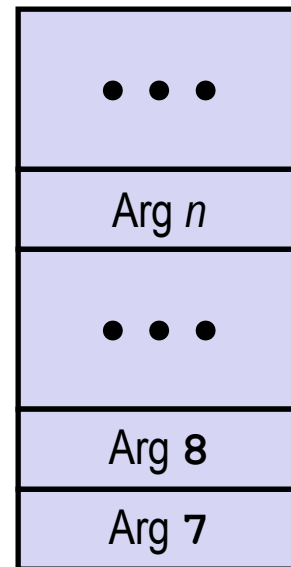
- First 6 arguments



- Return value



Stack

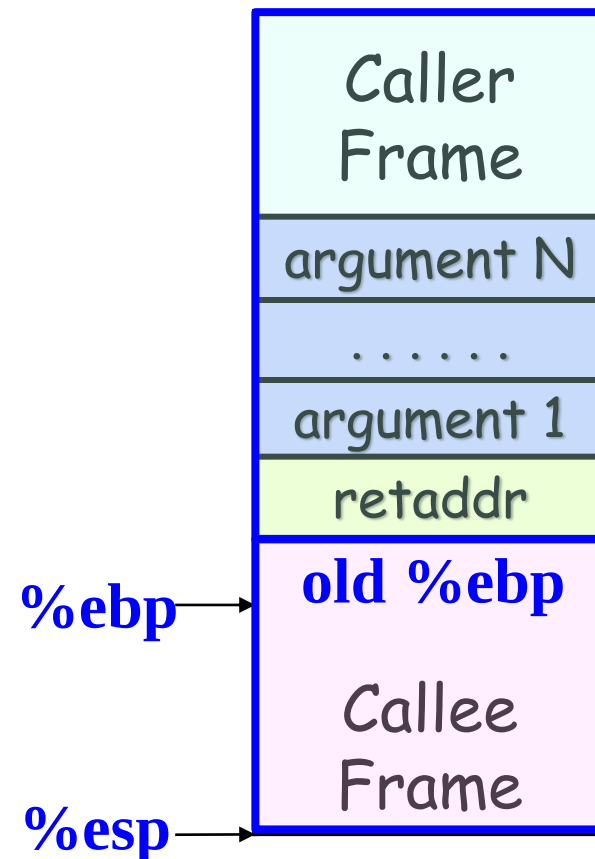


- Only allocate stack space when needed



传递数据：输入参数

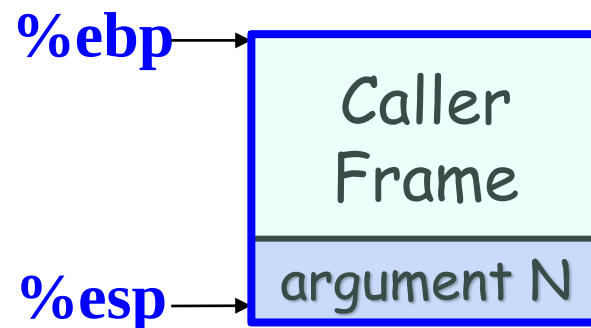
- Pushed by Caller
 - Saved in caller frame
 - Just upon of return address
 - From Nth to 1st (from **right to left**)
- Used by Callee
 - Relative to `%ebp`
 - Offset: $4 + 4*i + \%ebp$





传递数据：输入参数

push argument N



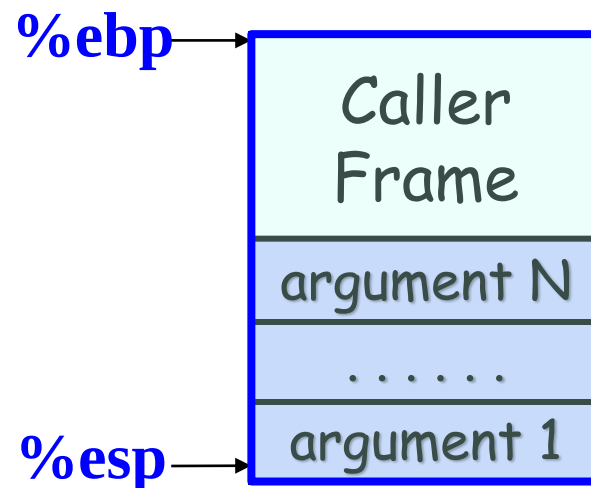


传递数据：输入参数

push argument N

...

push argument 1





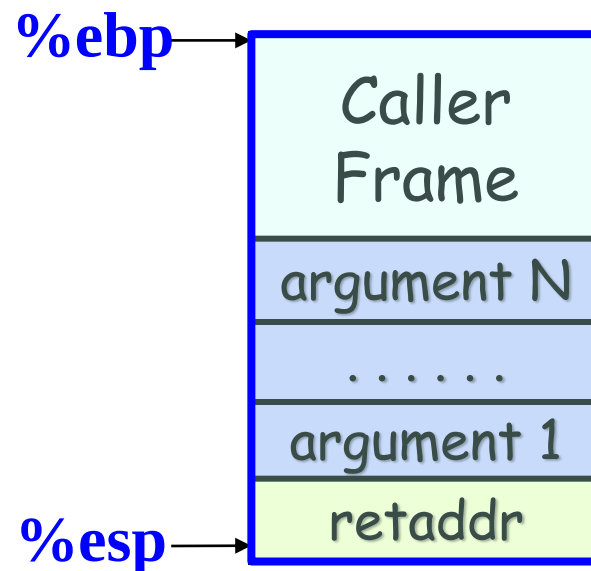
传递数据： 输入参数

push argument N

...

push argument 1

call callee





传递数据：输入参数

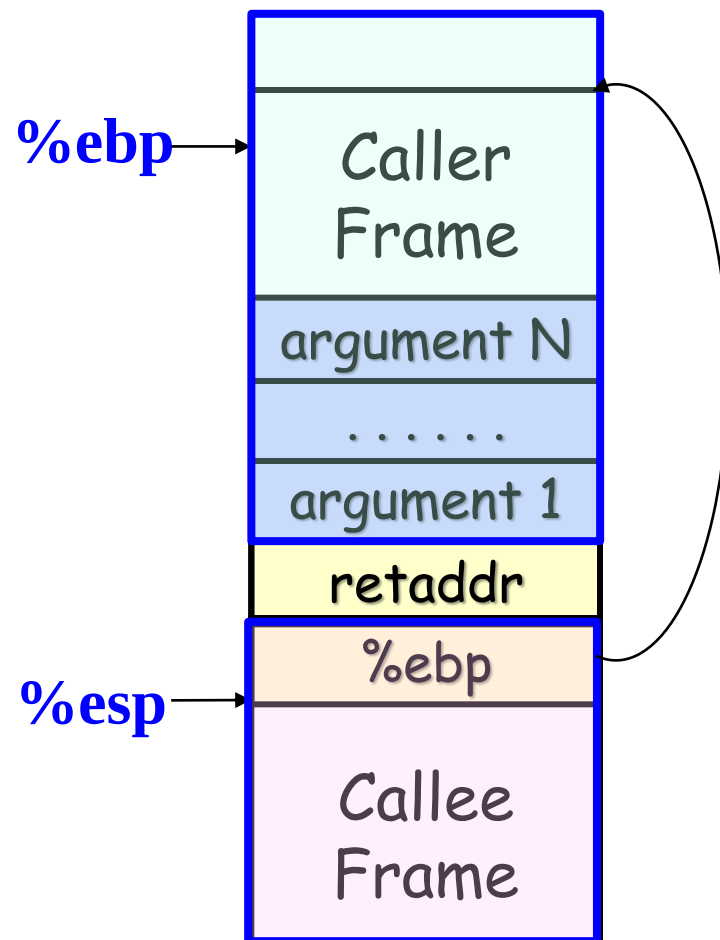
push argument N

...

push argument 1

call callee

push %ebp





传递数据： 输入参数

push argument N

...

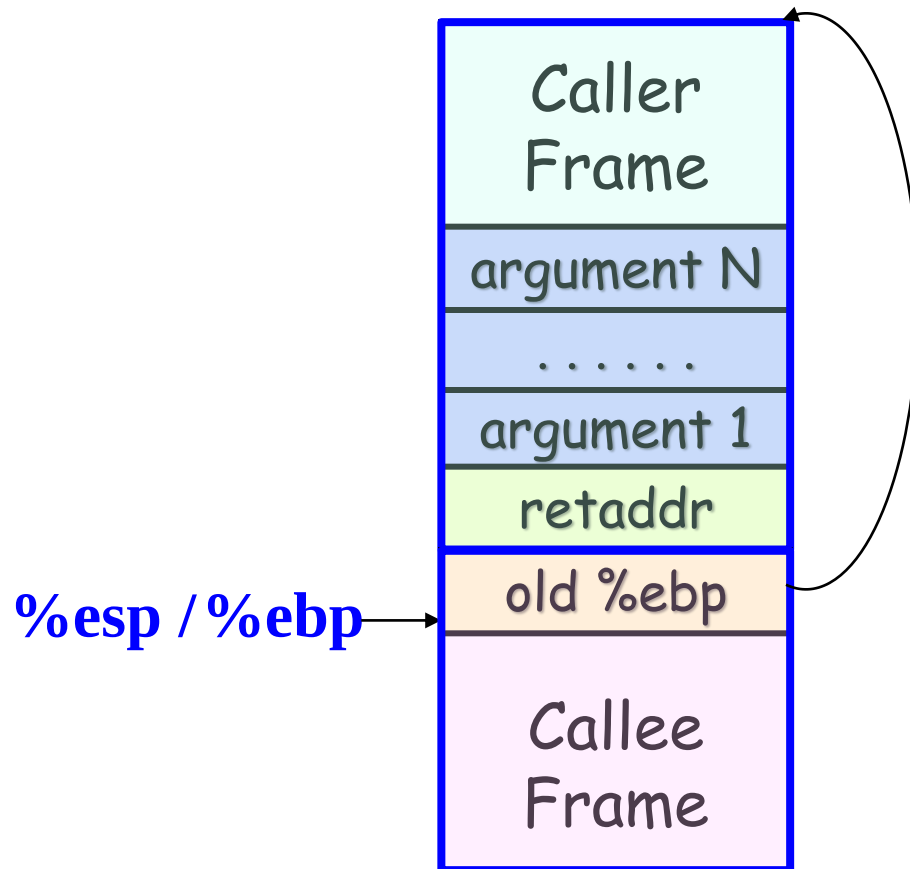
push argument 1

call callee

push %ebp

mov %esp, %ebp

...





传递数据： 输入参数

- X86 64位，传参数先用寄存器
 - %rdi, %rsi, %rdx, %rcx, %r8, %r9 用作函数参数，依次对应第1参数，第2参数....第6个参数（从左到右）
 - 剩余参数还和以前的算法一样，从右往左依次入栈
 - 例如func(p1, p2, p3, p4, p5, p6, p7, p8)
 - p1: %rdi, p2: %rsi, p3: %rdx, p4: %rcx, p5: %r8, p6: %r9
 - 栈：(低地址) $\leftarrow$ ret addr, p7, p8 $\leftarrow$ (高地址)



传递数据：返回值

- Specific register to keep the return value
 - `%eax / %rax` is used to pass the result of callee to caller



寄存器的保存

- 当函数 **yoo** 调用 **who**:
 - **yoo** is the *caller*
 - **who** is the *callee*
- 寄存器能否在被调用函数中直接使用?

yoo:

```
• • •  
movq $15213, %rdx  
call who  
addq %rdx, %rax  
• • •  
ret
```

who:

```
• • •  
subq $18213, %rdx  
• • •  
ret
```

- 寄存器 **%rdx** 的内容会在 **who** 中重写
- 可能引发问题 → something should be done!
 - 调用者和被调用者的协同



寄存器的保存

- 寄存器是所有过程共享的Single资源（竞争资源）
 - 只有一个函数可以使用
 - 仅考虑过程使用的寄存器
 - 分为caller 和 callee (必须遵守惯例)
 - ***“Caller Saved”***
 - Caller saves temporary values in its frame before the call
 - ***“Callee Saved”***
 - Callee saves temporary values in its frame before using
 - Callee restores them before returning to caller



x86-64 Linux Register Usage #1

- **%rax**

- Return value
- Also caller-saved
- Can be modified by procedure

- **%rdi, ..., %r9**

- Arguments
- Also caller-saved
- Can be modified by procedure

- **%r10, %r11**

- Caller-saved
- Can be modified by procedure

Return value

Arguments

Caller-saved
temporaries

%rax
%rdi
%rsi
%rdx
%rcx
%r8
%r9
%r10
%r11

- Caller must restore them if it tries to use them after calling



Caller-save Registers

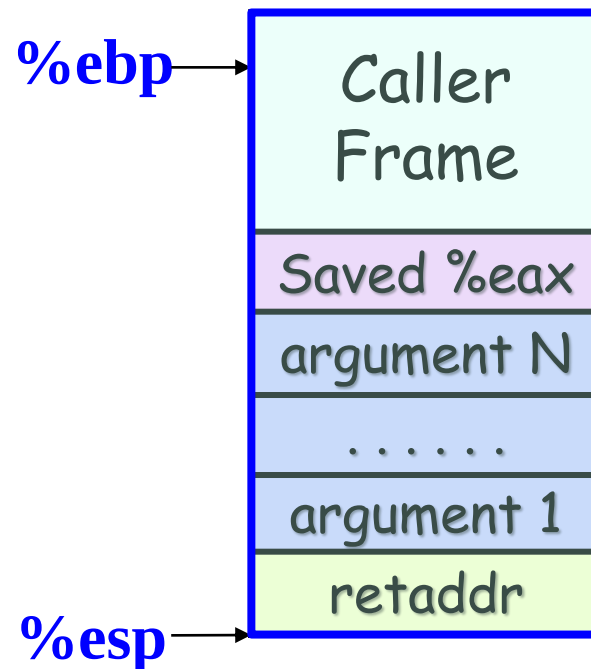
push %eax

push argument N

...

push argument 1

call callee





Calling Convention

- Callee-save registers

- %rbx, %rbp, %r12~%r14

- Saved by callee

- Caller can use these registers freely

- Callee must save them before using

- Callee must restore them before return

Callee-saved
Temporaries

Special

%rbx

%r12

%r13

%r14

%rbp

%rsp



Callee-save Registers

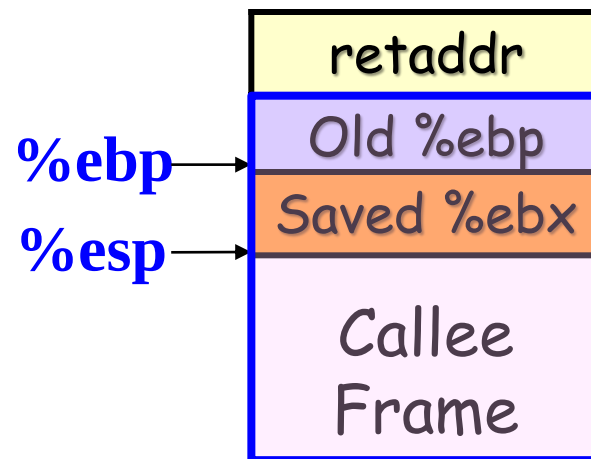
call callee

push %ebp

mov %esp, %ebp

push %ebx

...





局部变量

- Why not store local variables in registers ?
 - No enough registers
 - Array and structures (e.g., `a[2]`)
 - Need address (e.g., `&a`)



局部变量

- 分配

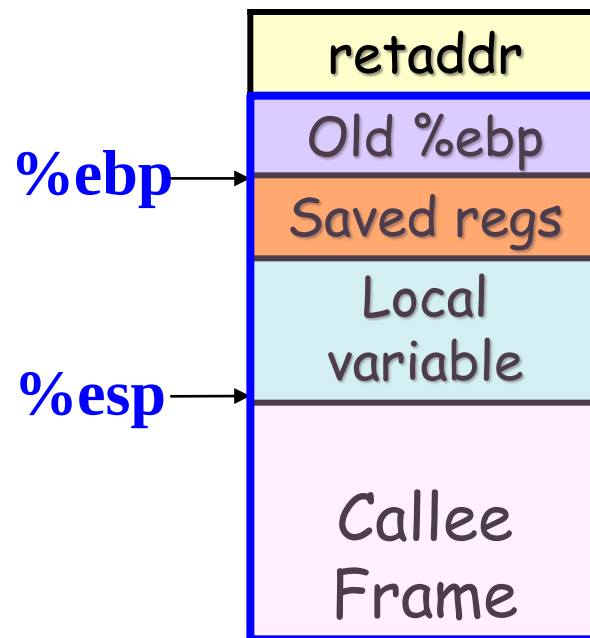
- Below saved regs or old %ebp
- `move/sub %esp, (e.g., subl $4, %esp)`

- 释放

- `move/add %esp, (e.g., addl $4, %esp)`

- 使用

- Relative to `%esp/%ebp`, (e.g., `movl %eax, 8(%esp)`)





Put it Together

%ebp

%esp

Caller
Frame



Put it Together

1. Save caller-save registers

(%rbx, %rbp, %r12~15, %rsp 以外的寄存器)

%ebp

Caller
Frame

%esp

caller-save
registers



Put it Together

1. Save caller-save registers
2. Push actual arguments
from right to left

%ebp

Caller
Frame

caller-save
registers
arguments
(n~1)

%esp



Put it Together

1. Save caller-save registers
2. Push actual arguments
from right to left
3. Call instruction
 - Save return address
 - Transfer control to callee

%ebp →

Caller
Frame

caller-save
registers
arguments
(n~1)

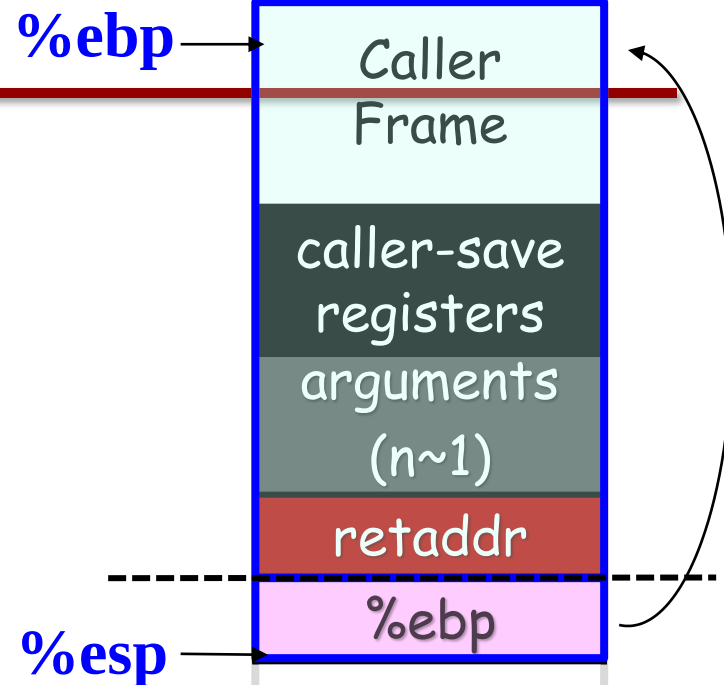
%esp →

retaddr



Put it Together

4. Save caller %ebp

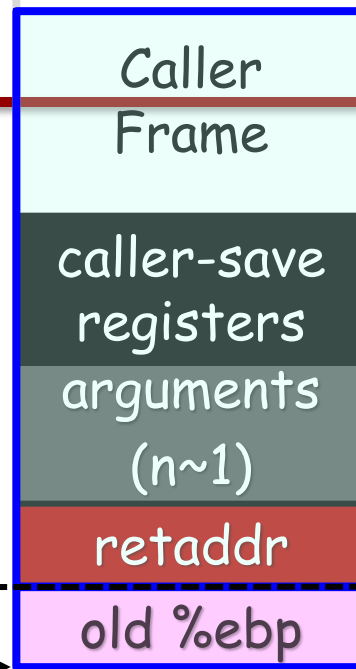




Put it Together

4. Save caller %ebp
5. Set callee %ebp

%esp / %ebp →





Put it Together

4. Save caller %ebp
5. Set callee %ebp
6. Save callee-save registers
(%rbx, %rbp, %r12~15)

%ebp →

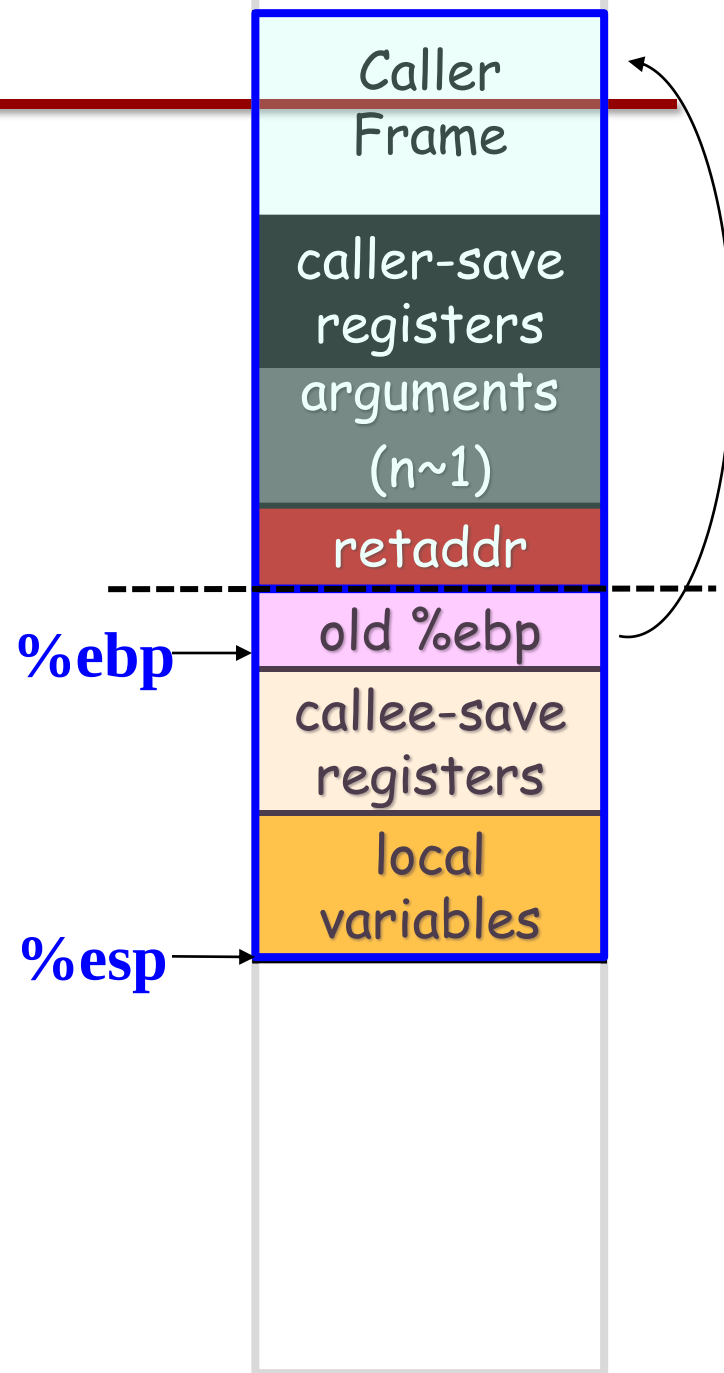
%esp →





Put it Together

4. Save caller %ebp
5. Set callee %ebp
6. Save callee-save registers
(%rbx, %rbp, %r12~15)
7. Allocate space for local
variable





Put it Together

...

n-4. save return value in %eax

%ebp

%esp

Caller
Frame

caller-save
registers

arguments
(n~1)

retaddr

old %ebp

callee-save
registers

local
variables





Put it Together

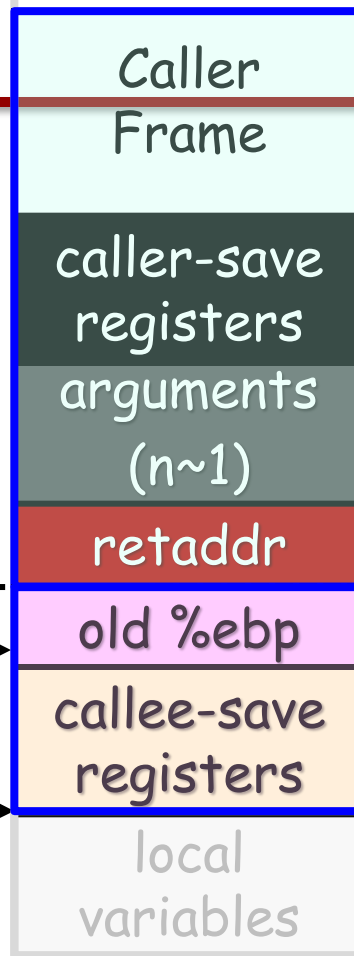
...

n-4. save return value in %eax

n-3. de-allocate local variable

%ebp

%esp





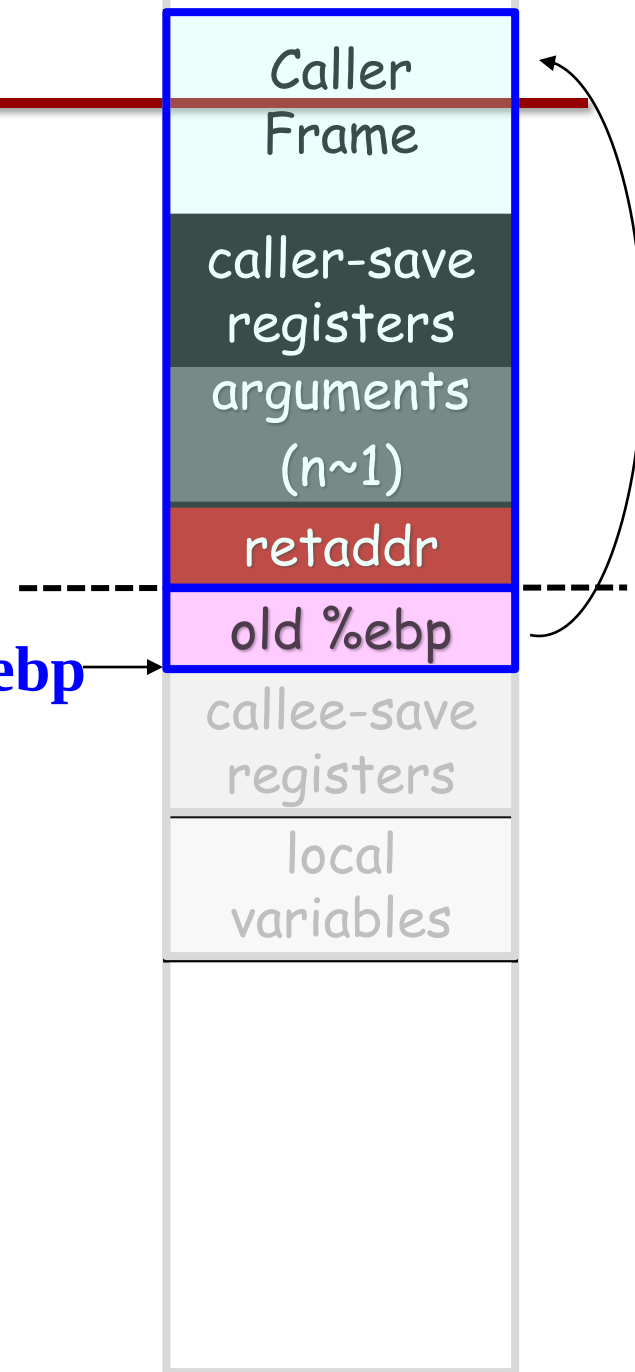
Put it Together

...

n-4. save return value in %eax

n-3. de-allocate local variable

n-2. Restore callee-save registers





Put it Together

...

n-4. save return value in %eax

n-3. de-allocate local variable

n-2. Restore callee-save registers

n-1. Restore caller %ebp

%ebp

Caller
Frame

caller-save
registers

arguments
(n~1)

retaddr

%esp

old %ebp

callee-save
registers

local
variables



Put it Together

...

n-4. save return value in %eax

n-3. de-allocate local variable

n-2. Restore callee-save registers

n-1. Restore caller %ebp

n. Ret instruction

- pop return address
- Transfer control to caller

%ebp

%esp

Caller
Frame

caller-save
registers
arguments
(n~1)

retaddr

old %ebp

callee-save
registers

local
variables



Example

```
1 int swap_add(int *xp, int *yp)
2 {
3     int x = *xp;
4     int y = *yp;
5
6     *xp = y;
7     *yp = x;
8     return x + y;
9 }
10
```



Example

```
11 int caller()  
12 {  
13     int arg1 = 534;  
14     int arg2 = 1057;  
15     int sum = swap_add(&arg1, &arg2);  
16     int diff = arg1 - arg2;  
17  
18     return sum * diff;  
19 }
```



Example

Before

int sum = swap_add(&arg1, &arg2);

Stack frame for caller

0	Saved %ebp	← %ebp
-4	1057(arg2)	
-8	534(arg1)	← %esp
-12		



Parameter Passing

1 `leal -4(%ebp),%eax`

Compute &arg2

2 `pushl %eax`

Push &arg2

Stack frame for caller

0	Saved %ebp	← %ebp
-4	1057(arg2)	
-8	534(arg1)	
-12	&arg2	← %esp



Parameter Passing

1 **leal -4(%ebp),%eax**

Compute &arg2

2 **pushl %eax**

Push &arg2

3 **leal -8(%ebp),%eax**

Compute &arg1

4 **pushl %eax**

Push &arg1

Stack frame for caller

0	Saved %ebp	← %ebp
-4	1057(arg2)	
-8	534(arg1)	
-12	&arg2	
-16	&arg1	← %esp



Call Instruction

1 **leal -4(%ebp),%eax**

Compute &arg2

2 **pushl %eax**

Push &arg2

3 **leal -8(%ebp),%eax**

Compute &arg1

4 **pushl %eax**

Push &arg1

5 **call swap_add**

Call the swap_add function

Stack frame for caller

0	Saved %ebp	← %ebp
-4	1057(arg2)	
-8	534(arg1)	
-12	&arg2	
-16	&arg1	
-20	Return Addr	← %esp



Setup code in swap_add

swap_add:

1 pushl %ebp

Save old %ebp

Stack frame for caller

0	Saved %ebp	← %ebp
-4	1057(arg2)	
-8	534(arg1)	
-12	yp(&arg2)	
-16	xp(&arg1)	
-20	Return Addr	
-24	Saved %ebp	← %esp



Setup code in swap_add

swap_add:

1 **pushl %ebp**

Save old %ebp

2 **movl %esp,%ebp**

Set %ebp as frame pointer

Stack frame for caller

24	Saved %ebp
20	1057(arg2)
16	534(arg1)
12	yp(&arg2)
8	xp(&arg1)
4	Return Addr
0	Saved %ebp

← %ebp

← %esp

Stack frame for swap_add



Setup code in swap_add

swap_add:

1 **pushl %ebp**

Save old %ebp

2 **movl %esp,%ebp**

Set %ebp as frame pointer

3 **pushl %ebx**

Save %ebx

Stack frame for caller

24	Saved %ebp	
20	1057(arg2)	
16	534(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

5 **movl 8(%ebp),%edx**

Get xp

%edx xp(=&arg1=%ebp+16)

Stack frame for caller

24	Saved %ebp	
20	1057(arg2)	
16	534(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

```
5  movl 8(%ebp),%edx
                                   Get xp
6  movl 12(%ebp),%ecx
                                   Get yp
```

%edx xp(=&arg1=%ebp+16)

%ecx yp(=&arg2=%ebp+20)

Stack frame for caller

24	Saved %ebp	
20	1057(arg2)	
16	534(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

```
5  movl 8(%ebp),%edx
                                   Get xp
6  movl 12(%ebp),%ecx
                                   Get yp
7  movl (%edx),%ebx
                                   Get x
8  movl (%ecx),%eax
                                   Get y
```

%edx xp(=&arg1=%ebp+16)

%ecx yp(=&arg2=%ebp+20)

%ebx 534

%eax 1057

Stack frame for caller

24	Saved %ebp	
20	1057(arg2)	
16	534(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

9 **movl %eax, (%edx)**

*Store y at *xp*

%edx **xp(=&arg1=%ebp+16)**

%ecx **yp(=&arg2=%ebp+20)**

%ebx **534**

%eax **1057**

Stack frame for caller

24	Saved %ebp	
20	1057(arg2)	
16	1057 (arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

9 **movl %eax, (%edx)**

*Store y at *xp*

10 **movl %ebx, (%ecx)**

*Store x at *yp*

%edx

xp(=&arg1=%ebp+16)

%ecx

yp(=&arg2=%ebp+20)

%ebx

534

%eax

1057

Stack frame for caller

24	Saved %ebp	
20	534(arg2)	
16	1057(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Body code in swap_add

9 **movl %eax, (%edx)**

*Store y at *xp*

10 **movl %ebx, (%ecx)**

*Store x at *yp*

11 **addl %ebx, %eax**

Set return value = x+y

%edx

xp(=&arg1=%ebp+16)

%ecx

yp(=&arg2=%ebp+20)

%ebx

534

%eax

1591

Stack frame for caller

24	Saved %ebp	
20	534(arg2)	
16	1057(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Finishing code in swap_add

12 **popl %ebx**

Restore %ebx

Stack frame for caller

24	Saved %ebp	
20	534(arg2)	
16	1057(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

%edx **xp(=&arg1=%ebp+16)**

%ecx **yp(=&arg2=%ebp+20)**

%ebx **original value**

%eax **1591**

Stack frame for swap_add



Finishing code in swap_add

12 popl %ebx *Restore %ebx*

13 movl %ebp, %esp
 Restore %esp

%edx **xp(=&arg1=%ebp+16)**

%ecx **yp(=&arg2=%ebp+20)**

%ebx **original value**

%eax **1591**

Stack frame for caller

24	Saved %ebp	
20	534(arg2)	
16	1057(arg1)	
12	yp(&arg2)	
8	xp(&arg1)	
4	Return Addr	
0	Saved %ebp	← %ebp
-4	Saved %ebx	← %esp

Stack frame for swap_add



Finishing code in swap_add

12 **popl %ebx** *Restore %ebx*

13 **movl %ebp, %esp**
 Restore %esp

14 **popl %ebp** *Restore %ebp*

%edx **xp(=&arg1=%ebp+16)**

%ecx **yp(=&arg2=%ebp+20)**

%ebx **original value**

%eax **1591**

Stack frame for caller

0	Saved %ebp	← %ebp
-4	534(arg2)	
-8	1057(arg1)	
-12	yp(&arg2)	
-16	xp(&arg1)	
-20	Return Addr	← %esp
	Saved %ebp	
	Saved %ebx	



Finishing code in swap_add

```
12 popl %ebx    Restore %ebx
13 movl %ebp, %esp    Restore %esp
14 popl %ebp    Restore %ebp
15 ret          Return to caller
```

- Call by value

%edx xp(=&arg1=%ebp+16)

%ecx yp(=&arg2=%ebp+20)

%ebx original value

%eax 1591

Stack frame for caller

0	Saved %ebp	← %ebp
-4	534(arg2)	
-8	1057(arg1)	
-12	yp(&arg2)	
-16	xp(&arg1)	← %esp
-20	Return Addr	
	Saved %ebp	
	Saved %ebx	



课堂练习

```
int proc(void) {  
    int x, y;  
    scanf("%x %x", &y, &x);  
    return x-y;  
}
```

过程proc开始时, esp值为0x800040, ebp值为0x00060, scanf输入值为0x46和0x53, “%x %x” 地址为0x300070.

- A. 第3行ebp的值被设为多少? **0x80003c;**
- B. 第4行esp的值被设为多少? **0x800014;**
- C. 局部变量x和y的地址是多少? **x:0x800038, y: 0x800034;**
- D. 画出scanf返回后的栈图
- E. 指出proc函数未使用的栈区的地址

0x800020~0x800033

■ GCC产生如下汇编代码

```
1.  proc  
2.  Pushl  %ebp  
3.  Movl   %esp, %ebp  
4.  Subl   $40, %esp  
5.  Leal   -4(%ebp), %eax  
6.  Movl   %eax, 8(%esp)  
7.  Leal   -8(%ebp), %eax  
8.  Movl   %eax, 4(%esp)  
9.  Movl   $.LCO, (%esp); 常量  
   字符串“%x %x”的地址  
10. Call  scanf  
11. Movl   -4(%ebp), %eax  
12. Subl   -8(%ebp), %eax  
13. Movl   %ebp, %esp,  
14. Popl   %ebp  
15. Ret
```



Recursive Function

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```



Recursive Function Terminal Case

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

Register	Use(s)	Type
%rdi	x	Argument
%rax	Return value	Return value



Recursive Function Register Save

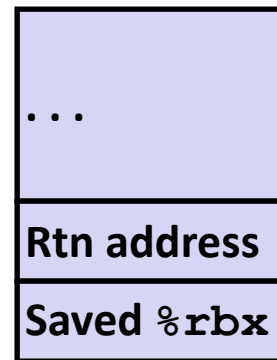
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

pcount_r:

```
movl    $0, %eax
testq   %rdi, %rdi
je      .L6
pushq   %rbx
movq    %rdi, %rbx
andl    $1, %ebx
shrq    %rdi
call    pcount_r
addq    %rbx, %rax
popq    %rbx
```

.L6:

rep; ret



Register	Use(s)	Type
%rdi	x	Argument



Recursive Function Call Setup

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

Register	Use(s)	Type
%rdi	x >> 1	Rec. argument
%rbx	x & 1	Callee-saved



Recursive Function Call

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Recursive call return value	



Recursive Function Result

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

Register	Use(s)	Type
%rbx	x & 1	Callee-saved
%rax	Return value	

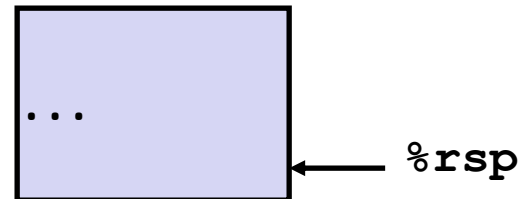


Recursive Function Completion

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

Register	Use(s)	Type
%rax	Return value	Return value





课堂练习

- 一个具有通用结构的C函数如下:

```
long rfun (unsigned long x) {  
    if ( x==0 )  
        return 0;  
    unsigned long nx = x/4;  
    long rv = rfun(nx);  
    return rv+x;  
}
```

- 请填写C语言代码中缺失的表达式;
rfun存储在被调用者保存寄存器
%rbx中的值是_____。

%rbx中的值是传入函数的参数x

GCC产生的汇编代码如下:

```
long rfun (unsigned long x)  
x in %rdi  
rfun:  
    pushq %rbx  
    movq %rdi, %rbx  
    movl $0, %eax  
    testq %rdi, %rdi  
    je    .L2  
    shrq $2, %rdi  
    call rfun  
    addq %rbx, %rax  
.L2:  
    popq %rbx  
    ret
```



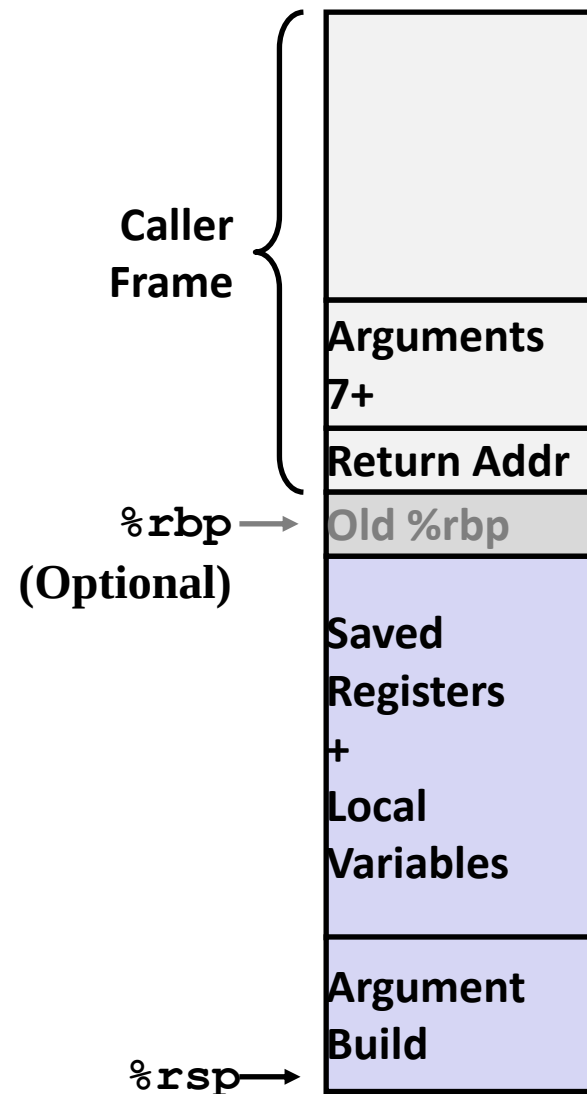
Observations About Recursion

- Handled Without Special Consideration
 - Stack frames mean that each function call has private storage
 - Saved registers & local variables
 - Saved return pointer
 - Register saving conventions prevent one function call from corrupting another's data
 - Unless the C code explicitly does so (e.g., buffer overflow in Lecture 9)
 - Stack discipline follows call / return pattern
 - If P calls Q, then Q returns before P
 - Last-In, First-Out
- Also works for mutual recursion
 - P calls Q; Q calls P



x86-64 Procedure Summary

- Important Points
 - Stack is the right data structure for procedure call / return
 - If P calls Q, then Q returns before P
- Recursion (& mutual recursion) handled by normal calling conventions
 - Can safely store values in local stack frame and in callee-saved registers
 - Put function arguments at top of stack
 - Result return in **%rax**
- Pointers are addresses of values
 - On stack or global

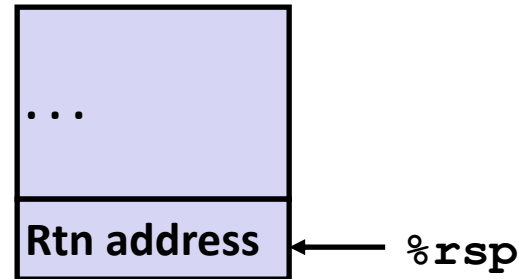




Callee-Saved Example #1

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

Initial Stack Structure



- **x** comes in register `%rdi`.
- We need `%rdi` for the call to `incr`.
- Where should be put **x**, so we can use it after the call to `incr`?

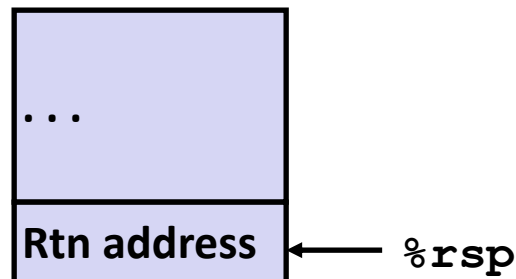


Callee-Saved Example #2

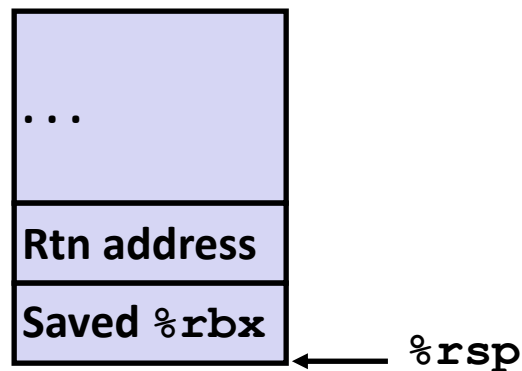
```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

Initial Stack Structure



Resulting Stack Structure



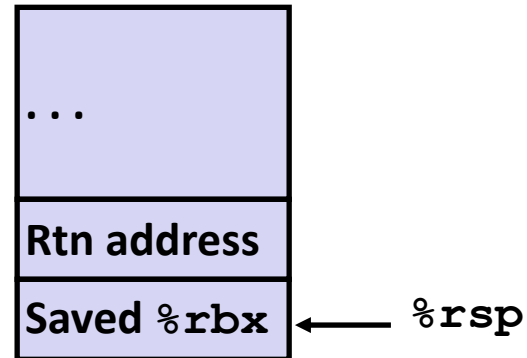


Callee-Saved Example #3

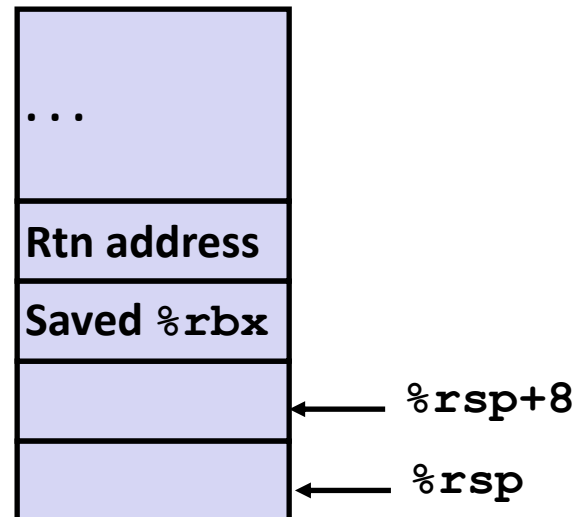
```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

Initial Stack Structure



Resulting Stack Structure



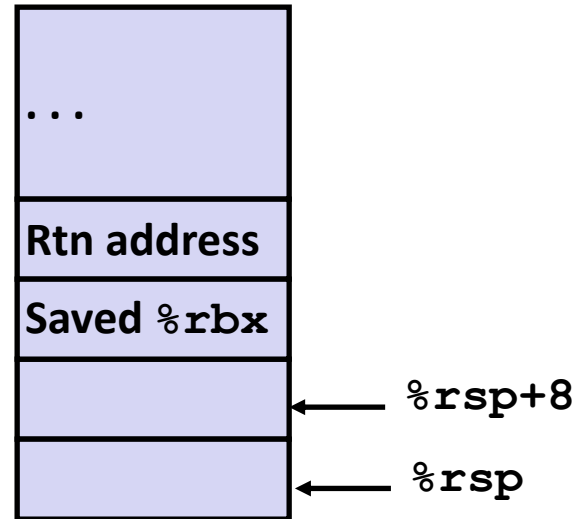


Callee-Saved Example #4

Stack Structure

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



- **x** is saved in **%rbx**,
a callee saved register

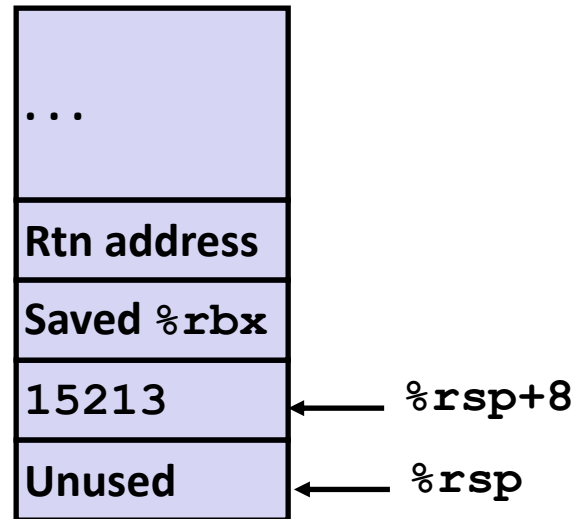


Callee-Saved Example #5

Stack Structure

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



- **x** is saved in **%rbx**,
a callee saved register

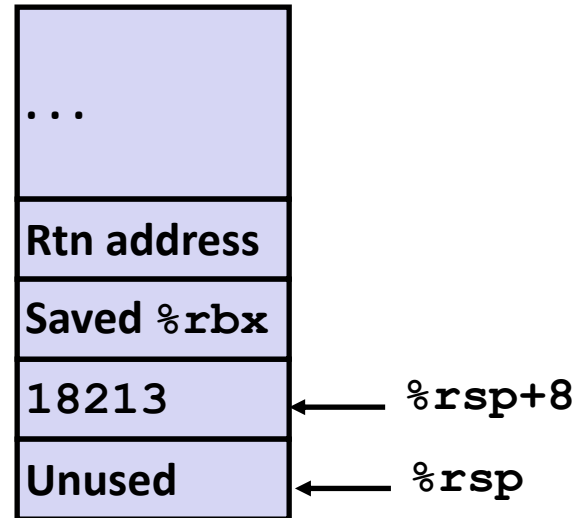


Callee-Saved Example #6

Stack Structure

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



Upon return from `incr`:

- `x` safe in `%rbx`
- Return val `v2` in `%rax`
- Compute `x+v2`:
`addq %rbx, %rax`

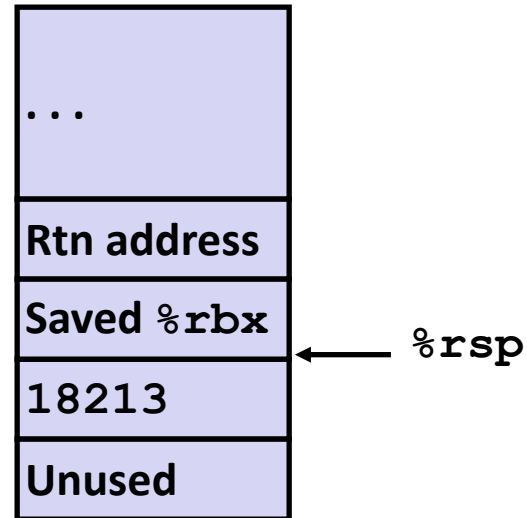


Callee-Saved Example #7

Stack Structure

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



- Return result in **%rax**

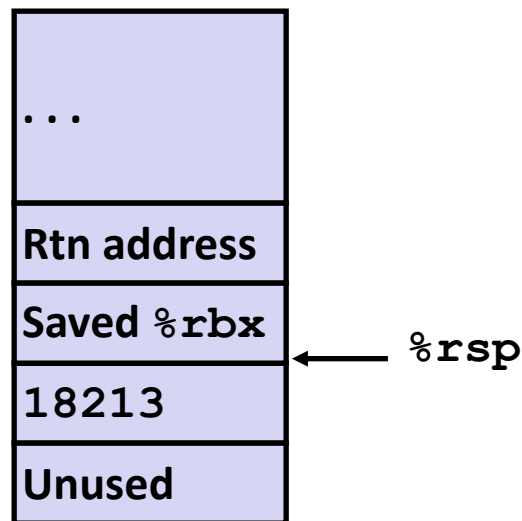


Callee-Saved Example #8

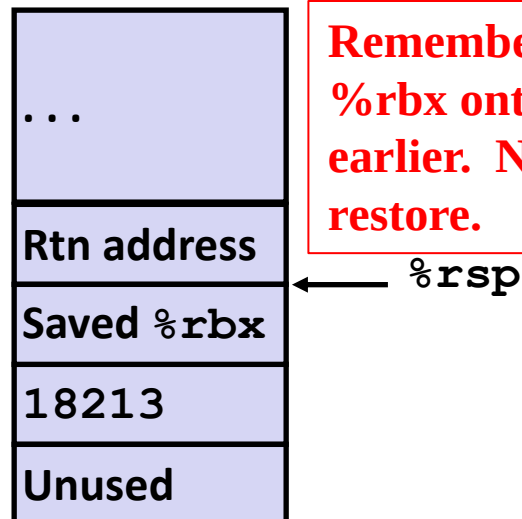
Initial Stack Structure

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



final Stack Structure



Remember: we saved %rbx onto stack earlier. Need to restore.